

EDT & TPU — Installation Guide

Sector7 USA LLC · EDIT/EDT and EDIT/TPU for macOS, Linux, Windows

Installation & licensing, July 2026

Table of contents

EDT & TPU — Installation Guide

- 1 What ships in the package
- 2 Executable matrix — pick the one for your machine
 - 2.1 EDT
 - 2.2 TPU
 - 2.3 How to choose
- 2.4 Per-platform setup notes
- 3 S7SECURITY — optional, and the one-time splash
 - 3.1 Where the flag file is written / read (search order)
 - 3.2 Turning the splash back on
 - 3.3 Legacy encrypted license blobs
 - 3.4 Quick verification

EDT & TPU — Installation Guide

Products: Sector7 EDIT/EDT ([edt](#)) and EDIT/TPU ([tpu](#)) **Applies to:** EDT V1.05-0, TPU V1.03-0 (July 2026)

Both editors are single-file native builds. Pick the executable that matches your machine (§2) and run — there is nothing else to install. A license file is **no longer required**; the optional **S7SECURITY** behaviour (a one-time startup splash) is described in §3.

1 What ships in the package

Each editor is delivered as a set of pre-built native executables, one per operating system and CPU architecture. There is nothing to compile and no installer: the executable is self-contained apart from the platform C/C++ runtime and the terminal library (curses), which are already present on a normal desktop/server install.

Every executable follows one naming convention:

```
<editor>_<opsy>_<chip>_<build>.exe
editor = edt | tpu
opsy   = w11 (Windows 11) | osx (macOS) | linux
chip   = x86 (Intel/AMD 64-bit) | arm86 (ARM 64-bit)
build  = release identity (SHOW VERSION), e.g. V1.05-0 (EDT), V1.03-0 (TPU)
```

The build number lets several releases coexist in one directory. This guide documents **EDT V1.05-0** and **TPU V1.03-0**; substitute the build number shown by `SHOW VERSION` for other releases.

The `.exe` suffix is used on **every** platform purely for a consistent naming convention — the macOS and Linux files are ordinary native binaries, not Windows programs. `file <name>` will confirm the real type.

2 Executable matrix — pick the one for your machine

2.1 EDT

File	Operating system	CPU / architecture	Notes
<code>edt_osx_arm86_V1.05-0.exe</code>	macOS 11 (Big Sur) or later	Apple Silicon, ARM 64-bit (M1/M2/M3/M4)	Mach-O native
<code>edt_linux_x86_V1.05-0.exe</code>	Linux (built on RHEL 8; runs on RHEL 8+/glibc 2.28+)	Intel/AMD 64-bit (x86-64)	ELF; needs <code>ncurses</code>
<code>edt_w11_x86_V1.05-0.exe</code>	Windows 10 / 11	Intel/AMD 64-bit (x86-64)	PE; PDCurses linked in
<code>edt_w11_arm86_V1.05-0.exe</code>	Windows 11	ARM 64-bit (Snapdragon / Windows-on-ARM)	PE (Aarch64)

2.2 TPU

File	Operating system	CPU / architecture	Notes
<code>tpu_osx_arm86_V1.03-0.exe</code>	macOS 11 (Big Sur) or later	Apple Silicon, ARM 64-bit	Mach-O native
<code>tpu_linux_x86_V1.03-0.exe</code>	Linux (built on RHEL 8; runs on RHEL 8+/glibc 2.28+)	Intel/AMD 64-bit (x86-64)	ELF; needs <code>ncurses</code>
<code>tpu_w11_x86_V1.03-0.exe</code>	Windows 10 / 11	Intel/AMD 64-bit (x86-64)	PE; PDCurses linked in
<code>tpu_w11_arm86_V1.03-0.exe</code>	Windows 11	ARM 64-bit (Windows-on-ARM)	PE (Aarch64)
<code>libtpu.dylib</code>	macOS 11+	Apple Silicon, ARM 64-bit	Shared library exposing the callable <code>TPU\$</code> API for embedding; not run directly

Not currently shipped (build from source with `g++ -std=c++17` if needed): macOS Intel (x86-64), Linux ARM64, and legacy 32-bit Windows.

2.3 How to choose

- **Mac (Apple Silicon):** `edt_osx_arm86_V1.05-0.exe` / `tpu_osx_arm86_V1.03-0.exe`.
- **Linux server or desktop, 64-bit Intel/AMD:** `edt_linux_x86_V1.05-0.exe` / `tpu_linux_x86_V1.03-0.exe`.
- **Windows 10/11 on a normal Intel/AMD PC:** `edt_w11_x86_V1.05-0.exe` / `tpu_w11_x86_V1.03-0.exe`.
- **Windows on ARM (Surface Pro X, Snapdragon laptops):** `edt_w11_arm86_V1.05-0.exe` / `tpu_w11_arm86_V1.03-0.exe`.

Run `file <name>` on macOS/Linux if you are unsure — it reports the exact architecture, e.g. `Mach-O 64-bit executable arm64` or `ELF 64-bit LSB executable, x86-64`.

2.4 Per-platform setup notes

macOS (Apple Silicon)

```
# Make it executable and (optionally) put it on your PATH:
chmod +x edt_osx_arm86_V1.05-0.exe
sudo cp edt_osx_arm86_V1.05-0.exe /usr/local/bin/edt
sudo cp tpu_osx_arm86_V1.03-0.exe /usr/local/bin/tpu
```

The binaries are unsigned, so Gatekeeper may block the first launch. Clear the quarantine attribute once:

```
xattr -d com.apple.quarantine /usr/local/bin/edt /usr/local/bin/tpu
```

curses is provided by the system; no extra packages are required.

Linux (RHEL 8 / x86-64)

```
chmod +x edt_linux_x86_V1.05-0.exe tpu_linux_x86_V1.03-0.exe
sudo cp edt_linux_x86_V1.05-0.exe /usr/local/bin/edt
sudo cp tpu_linux_x86_V1.03-0.exe /usr/local/bin/tpu
```

Requires the ncurses runtime (`libncursesw` / `libtinfo`). On RHEL/Rocky/Alma: `sudo dnf install ncurses` . On Debian/Ubuntu: `sudo apt install libncursesw6` . The build targets glibc 2.28 (RHEL 8), so it runs on RHEL 8 and any newer mainstream distribution.

Windows 10 / 11

Copy the matching `.exe` somewhere on your `PATH` (e.g. `C:\Sector7\bin`). PDCurses is statically linked, so no curses install is needed. If Windows reports a missing runtime DLL, install the current **Microsoft Visual C++ Redistributable** for your architecture.

3 S7SECURITY — optional, and the one-time splash

A license file is no longer required. `edt` and `tpu` start and run whether or not an `S7SECURITY.INI` file is present. There is nothing you must install for licensing, and there is no "security file not found" error any more — you can ignore this section entirely and just run the editor.

What `S7SECURITY.INI` now does is control a small, one-time branded **splash screen**:

- **First launch (no flag yet):** the editor shows the branded splash once for a few seconds (in TPU, until you press **RETURN**), then writes a small plain-text `S7SECURITY.INI` containing `splash=off` at the resolved path (see §3.1).
- **Every launch after that:** the editor reads `splash=off` and starts straight into the editor with no splash.

So the file is created automatically the first time you run either editor; you never have to create or install it yourself.

3.1 Where the flag file is written / read (search order)

1. **S7SECURITY environment variable** — if set and non-empty, it is treated as the **full path to the file** (any filename, any directory). This takes priority over everything else.
2. **Default file in your home directory**, used when **S7SECURITY** is not set:
3. macOS: `/Users/<you>/S7SECURITY.INI` (`$HOME/S7SECURITY.INI`)
4. Linux: `/home/<you>/S7SECURITY.INI` (`$HOME/S7SECURITY.INI`)
5. Windows: `C:\Users\<you>\S7SECURITY.INI` (`%USERPROFILE%\S7SECURITY.INI`)

If the location is not writable the editor still runs fine — the splash simply shows again next time instead of being suppressed.

3.2 Turning the splash back on

The auto-created file is plain text and looks like this:

```
# Sector7 editor settings (EDT / VX-TPU)
splash=off
```

To see the splash again, either delete the file or change the line to `splash=on` (any of `off/no/0/false` suppress it; anything else shows it). To suppress the splash for a single TPU run without touching the file, set `TPU_NO_SPLASH` in the environment.

3.3 Legacy encrypted license blobs

Earlier releases used an encrypted `S7SEC1` binary blob (issued by Sector7, carrying a licensee name/email and a splash duration). Those blobs are still read tolerantly: if one is present, its name/email is shown on the splash as `(c) Sector7 USA LLC <EDT|TPU> licensed to <name> <email>`. A blob is **not required**, and once the splash has been shown it is replaced by the plain `splash=off` file above, so an issued blob is effectively single-use. The `mksecurity` / `s7licdb` tools still generate these blobs for anyone who wants a branded splash.

3.4 Quick verification

```
# Should open the editor (line mode). QUIT to exit.
edt myfile.txt
# TPU with no file shows the splash once (press RETURN); afterwards it is
# suppressed and TPU starts straight into the editor.
tpu
```